

Continuous Integration and Deployment



☰ Continuous Integration and Deployment

☰ Logging in to Heroku Account

☰ GitHub

Continuous Integration and Deployment

We have successfully deployed the Iris Machine Learning application on the localhost server for predictions. In this lesson, we learn to use the GitHub repository for our ML applications and deploy applications to [Heroku](#) live servers and publish it.

Heroku is a cloud application platform. It provides a computer platform as a service, publishing and managing web applications, including ML applications. We will create a continuous integration and deployment pipeline in this lesson through GitHub and Heroku.

We will be using a few tools in this lesson:





Heroku



Sign up for a free account:

<https://www.heroku.com/>



Git Repository



Create a local Git Repository:

<https://git-scm.com/downloads>

Configure git on your local machine with your correct name, email address, and other information as required.



GitHub



Create a GitHub Account:

<https://github.com/join>

Python Virtual Environment

- First, create a new directory in Windows PowerShell and issue the following commands to create a virtual environment.

```
C:\Users\amf> mkdir deployHeroku  
  
C:\Users\amf> cd deployHeroku  
  
C:\Users\amf\deployHeroku> python -m venv cicd  
  
C:\Users\amf\deplyHeroku> .\cicd\scripts\activate
```

- Power Shell prompt changes to reflect (cicd) virtual environment (cicd) PS

```
C:\Users\amf\deployHeroku>
```

Next, clone the Iris classification repository below in your local machine:

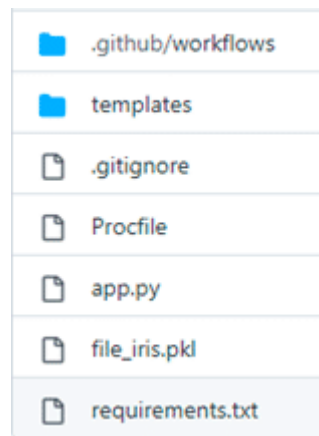
<http://github.com/amfarahani/deployHeroku>

Test the model on your local machine by issuing the command.

Start the localhost server and ensure the application is working (cicd):

```
\Users\amf\deployHeroku> flask run
```

The cloned repository has folders and files shown in the image:



Test the model on your local machine by issuing the command (cicd):

```
C:\Users\amf\ deployHeroku > flask run
```

Start the localhost server and ensure the application is working.

The next task is to deploy this model to Heroku Servers.

Login to Heroku Account

Heroku has a dashboard for managing projects. But first, we need to download and install Heroku CLI (Command Line Interface) in our virtual environment from

<https://devcenter.heroku.com/articles/heroku-cli>

Using Heroku CLI, login to Heroku at the Power Shell virtual environment prompt, type:

```
C:\Users\amf\deployHeroku> heroku login
```

Following the instructions on the screen.

After logging in, issue the command Heroku *create*, this creates a new Heroku application. In this example the application name is ana680-3 (cid):

```
C:\Users\amf\ML_heroku> heroku create ana680-3
»      Warning: heroku update available from 7.53.0 to 7.60.2.
Creating  ana680-3... done
```

<https://ana680-3.herokuapp.com/>

<https://git.heroku.com/ana680-3.git>

After this command, a Heroku application is created and deployed on Heroku servers. A link (URL) to the application is returned to the user to access the application. Click on the link to see the application running (which is an empty default application)

Logging in to Heroku Account

Heroku has a dashboard for managing projects. But first, we need to download and install Heroku CLI (Command Line Interface) in our virtual environment from

<https://devcenter.heroku.com/articles/heroku-cli>

Using Heroku CLI, login to Heroku at the Power Shell virtual environment prompt, type:

```
C:\Users\amf\deployHeroku> heroku login
```

Following the instructions on the screen.

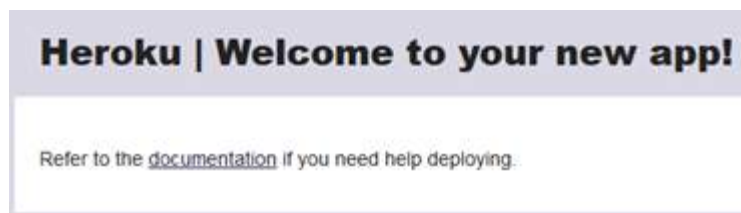
After logging in, issue the command Heroku *create*, this creates a new Heroku application. In this example the application name is ana680-3 (cicd):

```
C:\Users\amf\ML_heroku> heroku create ana680-3
»      Warning: heroku update available from 7.53.0 to 7.60.2.
```

```
Creating  ana680-3... done
```

<https://git.heroku.com/ana680-3.git>

After this command, a Heroku application is created and deployed on Heroku servers. A link (URL) to the application is returned to the user to access the application. Click on the link to see the application running (which is an empty default application)



The application site is:

<https://ana680-3.herokuapp.com/>

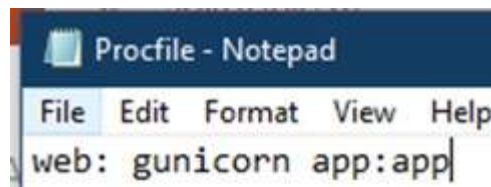
Heroku has its git repository to which the ML application will be pushed.

You can verify the creation of the new application on the Heroku site by going to your Heroku account online and clicking on the dashboard.



Procfile

Heroku requires a *Procfile* that specifies the commands that are executed on startup. The cloned repository already has a *Procfile* that you may open and examine. It has two commands the web server, which is **gunicorn**, and the application to run, which is **app**.



As the final step, the git project on the local machine is pushed to Heroku repository (cicd).

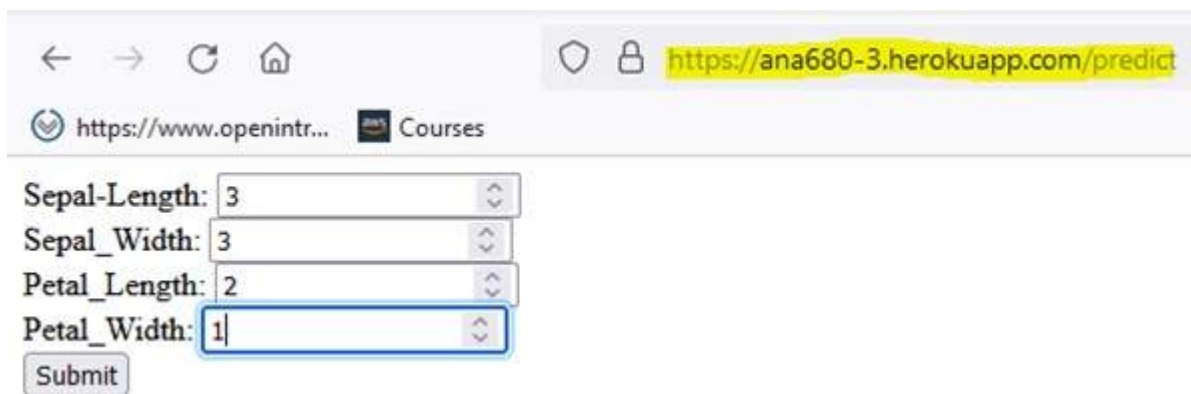
```
C:\Users\amf\gitProjects\deployHeroku> git push https://git.heroku.com/ana680-3
```

Install gunicorn in the virtual environment (cicd)

```
C:\Users\amf\gitProjects\deployHeroku> pip install gunicorn
```

Once the ML project is pushed to the Heroku servers, it will be posted at Heroku project site

<https://ana680-3.herokuapp.com/>



The screenshot shows a web browser window with the address bar displaying <https://ana680-3.herokuapp.com/predict>. The page contains a form with four input fields: 'Sepal-Length' with value 3, 'Sepal_Width' with value 3, 'Petal_Length' with value 2, and 'Petal_Width' with value 1. A 'Submit' button is located below the inputs. The browser's address bar also shows a tab for 'https://www.openintr... Courses'.

Notice the URL is not the local machine. The model is running on Heroku server and made public.

So far, we have pushed our ML project from our local machine to Heroku git repository and deployed it on the Heroku server. In the current setup any changes we make to the ML model and commit to our local repository will have to be then pushed to Heroku repository.

Next, we will automate this process through Continuous Integration and Deployment.

GitHub

First, go to your GitHub and create a new GitHub repository. Call it *deployHeroku*.
Next, add the new repository as a remote repository.

```
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
```

The code in the command above is an authentication code given by your GitHub.

```
C:\Users\amf\gitProjects\deployHeroku\templates> git remote add  
heroku2https://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@github.com/amfarah
```

Next, push your local ML project repository to this new GitHub repository.

```
C:\Users\amf\gitProjects\deployHeroku\templates> git push  
https://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@github.com/amfarahani/test
```

Open the file myFLASK1.yml file in `.github\workflow` in the local repository. This file is the GitHub Actions YAML file that guides the CI/CD process.

This file describes the package needed to build the ML applications on the server (in this case, the Heroku server). The server will automatically install all the required tools and packages listed in this file to build the application.

Ensure your application name and email address are your (in Heroku) and email address.

The HEROKU_API_KEY must be set up in your GitHub testHero repository.

```
- name: deploy to Heroku  
  uses: akhileshns/heroku-deploy@v3.12.12 # This is the action  
  with:  
    heroku_api_key: ${secrets.HEROKU_API_KEY}  
    heroku_app_name: "ana680-3" #Must be unique in Heroku  
    heroku_email: "afarahan@nu.edu"
```

Find the API_KEY on the Heroku site. It is listed under Account Settings.

In testHero repository, click on repository settings, select Secrets/Actions, then click on create a New Secret button. For the secret's Name field, enter HEROKY_API_KEY and for the Value, enter the key you obtained from Heroku. This setup allows your GitHub repository to communicate with Heroku.

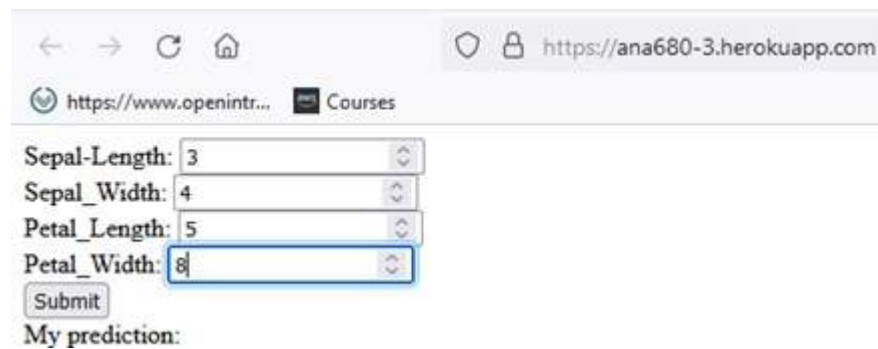


Going back to the *myFLASK1.yml* file, note that the file has an event called on: [push], which indicates that the GitHub actions file is executed every time a push is applied to the repository. So the push is needed when a commit is made in the git repository. This, in turn, starts the deployment of the changes to the Heroku server, which is Continuous Integration and Deployment.

Next, push your local ML project repository to your new GitHub repository.

```
C:\Users\amf\gitProjects\deployHeroku\templates> git push  
https://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@github.com/amfarahani/test
```

Once the ML project is pushed to Heroku, it will be at Heroku project site <https://ana680-3.herokuapp.com/> . Note that the URL is no longer the localhost.



← → ↻ 🏠 <https://ana680-3.herokuapp.com>

🔒 <https://www.openintr...> Courses

Sepal-Length: 3

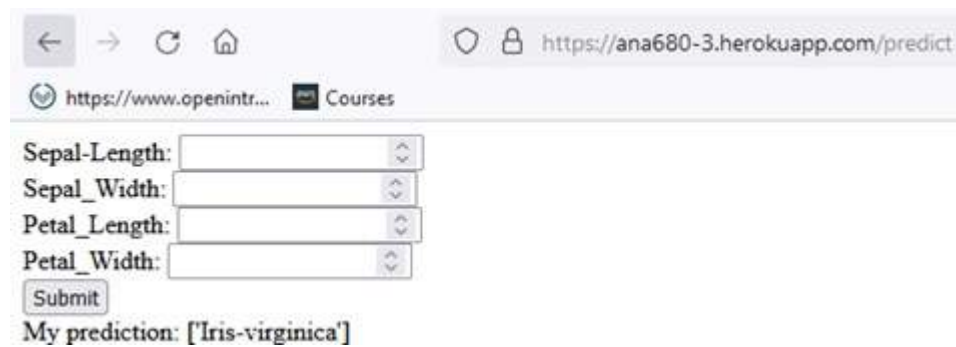
Sepal_Width: 4

Petal_Length: 5

Petal_Width: 8

Submit

My prediction:



← → ↻ 🏠 <https://ana680-3.herokuapp.com/predict>

🔒 <https://www.openintr...> Courses

Sepal-Length:

Sepal_Width:

Petal_Length:

Petal_Width:

Submit

My prediction: ['Iris-virginica']

Test Continuous Integration and Deployment:

Try changing some text in the *index.html* file, commit and push the revision to the testHero repository. Verify the change in the application, Heroku automatically redeploys the application with all the changes.